Regime Detection using Sliced Wasserstein k-means clustering with Monte Carlo

Oscar Peyron
ADIA Lab - University College London

February 2026

1 Abstract

This thesis investigates the topic of Regime Detection using an interpretable Machine Learning algorithm. In particular, the focus of this thesis is to produce a tractable algorithm for detecting regimes in financial markets based on multi-dimensional time series. This thesis, provides an enhanced algorithm to cluster the multi-dimensional returns distributions. Depending, on the the characteristics of the distributions, the algorithm can infer which regime the market is in without pre-specified assumptions on the returns distribution. Specifically, it uses the Wasserstein distance as a metric to compare different regimes. In higher-dimensional settings, one can extend this concept using the Sliced Wasserstein distance and use an algorithm based on this metric to cluster different market regimes (Sliced Wasserstein k-means clustering algorithm). The following thesis is structured as follows believed to be the most explanatory and logical way to present the research.

1.1 Experiment 1: Theoretical Framework for Wasserstein Distance/Sliced Wasserstein (SW) Distance

This chapter will focus on the theoretical framework to understand the Wasserstein distance and its extension to the Sliced Wasserstein distance. Moreover, this section will provide insights on the practical implementation of the above-mentionned algorithms and limitations to the current SW clustering implementations in the literature. Notably, the low generalizable nature of using pre-defined projection vectors θ^l for the Sliced Wasserstein can be solved by generating them using the UnifOrtho method [3]. It provides the following advantages compared to standard Monte Carlo methods.

- **Variance reduction via Negative Negative Dependance**
- **Superior performance in high dimensions** : The UnifOrtho is therefore perfectly adapted to working with multi-dimensional time series for the SW k-means clustering algorithm.
- **Clustering Stability**

Some useful theoretical results on variance reduction and computational cost on the UnifOrtho method will therefore be provided to support the choice of this method for the future sections.

1.2 Experiment 2: Design methodology for Monte Carlo for SW - Synthetic Data Testing

The focus of this chapter will be on testing the algorithm on syntethic data to understand the behaviour of the algorithm and its performance in different settings. The algorithm will be tested on 3 dimensional data followed by 4, 5 dimensional. Performance between original and UniOrtho method will be compared and tests will span from data with multiple correlation factors and with diffrent target number of clusters. Finally, in terms of time-horizon, the study will focus on the equivalent of 20 years synthetic data with 7 observations a day (252 trading days a year) to be as close as possible to real world data.

Different performance metrics will be used for the regime detection :

- Total Accuracy (TA)

- Mean centroid-centroid distance (MCCD)
- Point-to-centroid distance (PCD)

The synthetic data will be generated similarly to previous papers [1] and [2] using Geometric Brownian Motion (GBM) paths $S_t/S_{t-1} = \exp(r_t^S)$ with $S_t = (S_t^1, \ldots, S_t^d)$ and $r_t^S = (r_t^{S^{(1)}}, \ldots, r_t^{S^{(d)}})$ where :

$$r_t^{S^{(i)}} \sim \mathcal{N}\left((\mu - \sigma^2/2)\mathbf{1}dt, \mathbf{\Sigma}dt\right) \tag{1}$$

. with

$$\Theta = (\mu, \sigma^2) \tag{2}$$

And the covariance matrix $\mathbf{\Sigma}$ is defined as follows :

$$\Sigma_{ij} = \sigma^2 \left(\delta_{ij} + (1 - \delta_{ij})\rho\right) \tag{3}$$

The δ_{ij} is the Kronecker delta, σ^2 is the variance of the returns and ρ is the correlation between the different assets.

Assuming, we want to cluster two different regimes (K=2), we define, as in previous papers [2], Bull and Bear regimes with the following parametrization :

- $\Theta_{\text{bull}} = (0.02, 0.2)$
- $\Theta_{\text{bear}} = (-0.02, 0.3)$

For K = 3, similarly to [2] we will consider the *moon-shape structure* generated using `datasets.make_moons` function from the `sklearn` library.

This is a summary similary to the one provided in [2] for the different market settings on which we are going to test the sWk-means clustering algorithm with UnifOrtho projections :

Table 1: Summary of synthetic dataset parameters with $d \in \{3, 4, 5\}$

Type	Regime Bull	Regime Bear	Regime III (Bear/Moon)
Type A (K=2)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = +1/2$	N/A
Type B (K=2)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bull}}; \rho = -1/2$	N/A
Type C (K=3)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = -1/2$
Type D (K=3)	$\Theta_{\text{bull}}; \rho = +1/2$	$\Theta_{\text{bear}}; \rho = +1/2$	$\Theta_{\text{moon}}; \rho = +1/2$

1.3 Experiment 3: Application of Monte Carlo SW for Real Data and Analysis

Finally, the chapter will focus on the application of the novel algorithm on real world data sets for different types of financial markets. As above, the time span, wil focus on 15 to 20 years of data. The target data sets include the following :

- FX Spot Series Data (4D)

  EURUSD, AUDUSD, USDJPY, USDCHF, USDCAD (Major Currency Cluster) (15 min data)

- Equity Series Data (3D)

  S&P 500, NASDAQ, Dow Jones (US Equity Cluster) (daily data)

  CAC 40, DAX40, FTSE 100 (European Equity Cluster) (daily data)

  Nikkei 225, Hang Seng, Kospi 200 (Asian Equity Cluster) (daily data)

- Commodity Series Data (4D and 5D)

  WTI Crude Oil, Brent Crude Oil, Natural Gas, Heating Oil.

  Copper Aluminum, Zinc, Nickel, Lead.

1.4 Experiment 4: Performance analysis of regime detection using trading strategies

- Long/Short strategy based on majority regime detection signals
- Long/Short strategy based on Implied probability signals (using the distance to the centroids as a proxy for the confidence of the regime detection)

1.4.1 Standard SW k-means trading strategy

Pseudo code for this strategy is provided here :

Algorithm 1: LONG_STRAT_UNIFORTHO — Binary Regime Strategy

Input: Initial capital C_0,;
number of simulations N_S,;
price matrix $S \in \mathbb{R}^{T\times d}$,;
trimming parameters h_1, h_2,;
window size w,;
number of clusters K,;
risk metric m,;
majority look-back ℓ
Output: Portfolio value path V

$R \leftarrow \mathrm{PctReturns}(S)$; ▷ $R_t = S_t/S_{t-1} - 1$
$\theta \leftarrow \mathbf{1}_d/d$; ▷ `equal-weight vector`
$r^{(\mathrm{pf})} \leftarrow R\,\theta$; ▷ `portfolio return series`
$V \leftarrow [C_0]$;
$n \leftarrow \lfloor T/w \rfloor$;
for $i = 0$ **to** $n-2$ **do**
  $s \leftarrow i \cdot w, \quad e \leftarrow (i+1)\cdot w$;
  $W \leftarrow S[s:e]$; ▷ `current analysis window`
  ▷ `Step 1 -- Wasserstein-based regime detection`
  $\hat{F},\ \boldsymbol{\mu},\ \boldsymbol{\lambda} \leftarrow \mathrm{MAXMCCD_UNIFORTHO}(N_S,\ W,\ K,\ L,\ \varepsilon,\ h_1,\ h_2,\ m)$;
  ▷ `Step 2 -- Majority-vote regime classification`
  if $\ell > |\boldsymbol{\lambda}|$ **then**
    $\hat{k} \leftarrow \arg\max_k\ \mathrm{count}(\boldsymbol{\lambda} = k)$;
  else
    $\hat{k} \leftarrow \arg\max_k\ \mathrm{count}(\boldsymbol{\lambda}_{[-\ell:]} = k)$;
  ▷ `Step 3 -- Generate binary signal`
  if $\hat{k} = 1$ **then**
    $\sigma \leftarrow +1$; ▷ `Bullish ⇒ Long`
  else
    $\sigma \leftarrow -1$; ▷ `Bearish ⇒ Short`
  ▷ `Step 4 -- Compound portfolio over next window`
  for *each* $r_t \in r^{(\mathrm{pf})}[e:e+w]$ **do**
    $V.\mathrm{append}\Big(V[-1]\ \times\ \big(1+\sigma\, r_t\big)\Big)$;
return V

1.4.2 Implied Probability SW k-means trading strategy

Theoretical Background We use implied probability measure to compute wether on thinks the market is going to become bear or bull.

Likelihood Function It uses SW-distance to every centroid to every centroid and then convert the likelihood via a softmax over the negative distances. We introduce a temperature parameter τ

$$\mathcal{L}(\text{regime } k \mid x_m) = \frac{e^{-d(x_m, c_k)/\tau}}{\sum_j e^{-d(x_m, c_j)/\tau}}$$

Empirical Transition Matrix Markov Prior A market in regime i is highly likely to stay in regime i tomorrow. We model this as a first-order Markov Chain using the historical labels. Probability of switching from i to j is:

$$\pi_{i,j} = P(Z_m = j \mid Z_{m-1} = i) = \frac{\text{Count}(i \to j)}{\sum_{x=1}^{K} \text{Count}(i \to x)}$$

If the market was $z_{current}$ our prior belief that state M is going to be at regime k is :

$$\mathbb{P}_{prior}(k) = \pi_{z_{current},k}$$

Then we simply use Bayes rule to derive the following :

$$\mathbb{P}_{Bayes}(k) = \frac{\mathbb{P}_{prior}(k) \cdot \mathcal{L}(k \mid X_m)}{\sum_{j=1}^{K} \mathbb{P}_{prior}(j) \cdot \mathcal{L}(j \mid X_m)}$$

Switch Probability Finally the switch probability does the following :

$$\mathbb{P}_{switch} = 1 - \mathbb{P}_{Bayes}(z_{current})$$

Use Gradient One may also want to take into account the fact that one is moving away or getting closer to a certain regime.

We consider a lookback window of size ω, and one computes the linear trend slope of the Wasserstein distance to each centroid k. Let $t \in \{1, 2, \ldots, \omega\}$ We get using OLS, β_k:

$$\beta_k = \frac{\text{Covariance}(t,\, \mathcal{W}_2(X_{M-w+t},\, C_k))}{\text{Variance}(t)}$$

- if $\beta_k < 0$ the distance to regime k is shrinking (we are getting close to regime k)
- if $\beta_k > 0$ the distance to regime k is growing (we are drifting away from regime k)

We apply softmax to get a probability:

$$P_{Grad}(k) = \frac{\exp\left(-\frac{\beta_k}{\tau}\right)}{\sum_{j=1}^{K} \exp\left(-\frac{\beta_j}{\tau}\right)}$$

The final switch probability using gradient is therefore:

$$\mathbb{P}_{switch} = 1 - [(1-\alpha)\mathbb{P}_{Bayes}(z_{current}) + \alpha \mathbb{P}_{Grad}(z_{current})]$$

with $\alpha \in [0:1]$ a weight.

Algorithm 2: LONG_STRAT_IMPLIED — Implied-Probability Regime Strategy

Input: Initial capital C_0,;
simulations N_S,;
price matrix $S \in \mathbb{R}^{T\times d}$,;
trimming h_1, h_2,;
window w,;
clusters K,;
metric m,;
signal type $\tau \in \{\texttt{continuous}, \texttt{hysteresis}, \texttt{conviction}\}$,;
entry threshold α_e,;
hold threshold α_h,;
look-back ℓ,;
gradient flag and weight $(\mathtt{g}, \omega_g)$
Output: Portfolio value path V,;
signal history Σ

$R \leftarrow \mathrm{PctReturns}(S), \quad \theta \leftarrow \mathbf{1}_d/d, \quad r^{(\mathrm{pf})} \leftarrow R\,\theta;$
$V \leftarrow [C_0], \quad \Sigma \leftarrow [\,], \quad n \leftarrow \lfloor T/w \rfloor;$
for $i = 0$ **to** $n-2$ **do**
  $s \leftarrow i \cdot w, \quad e \leftarrow (i+1)\cdot w, \quad W \leftarrow S[s:e];$
  ▷ `Step 1 -- Regime detection (same as Algorithm 1)`
  $\hat{F},\ \boldsymbol{\mu},\ \boldsymbol{\lambda} \leftarrow \textsc{MaxMCCD_UnifOrtho}(N_S,\ W,\ K,\ L,\ \varepsilon,\ h_1,\ h_2,\ m);$
  ▷ `Step 2 -- Implied transition probabilities`
  $\mathbf{P},\ p_{\mathrm{switch}},\ \mathbf{T},\ \boldsymbol{\pi} \leftarrow \textsc{ComputeImpliedProba}(\hat{F},\ \boldsymbol{\mu},\ \boldsymbol{\lambda},\ \ell,\ \mathtt{g},\ \omega_g);$
  $\hat{k} \leftarrow \arg\max_k\ \mathrm{count}(\boldsymbol{\lambda}_{[-h_2:]} = k);$
  ▷ `Step 3 -- Signal generation (depends on` τ`)`
  if $\tau =$ *continuous* **then**
    $\sigma \leftarrow \pi_{\mathrm{bull}} - \pi_{\mathrm{bear}};$
    if $|\sigma| < \delta$ **then**
      $\sigma \leftarrow 0$
    ; ▷ `dead zone`
  else if $\tau =$ *hysteresis* **then**
    $\sigma_{\mathrm{prev}} \leftarrow \Sigma[-1]$ **or** 0;
    if $\hat{k} = \mathit{Bull}$ **then**
      if $p_{switch} \geq \alpha_e$ ***and*** $\sigma_{prev} \geq 0$ **then** $\sigma \leftarrow -1$;
      else if $p_{switch} < \alpha_h$ ***and*** $\sigma_{prev} < 0$ **then** $\sigma \leftarrow +1$;
      else $\sigma \leftarrow 0$;
    else ▷ $\hat{k} =$ `Bear`
      if $p_{switch} \geq \alpha_e$ ***and*** $\sigma_{prev} \leq 0$ **then** $\sigma \leftarrow +1$;
      else if $p_{switch} < \alpha_h$ ***and*** $\sigma_{prev} > 0$ **then** $\sigma \leftarrow -1$;
      else $\sigma \leftarrow 0$;
  else if $\tau =$ *conviction* **then**
    $d_k \leftarrow \begin{cases} +1 & \hat{k} = \mathrm{Bull} \\ -1 & \hat{k} = \mathrm{Bear} \end{cases};$
    $c \leftarrow 1 - 1.5\,p_{\mathrm{switch}}\ ;$ ▷ `conviction decays with switch risk`
    $\sigma \leftarrow d_k \cdot c;$
  $\Sigma.\mathrm{append}(\sigma);$
  ▷ `Step 4 -- Compound portfolio over next window`
  for *each* $r_t \in r^{(\mathrm{pf})}[e:e+w]$ **do**
    $V.\mathrm{append}\Big(V[-1]\ \times\ \big(1 + \sigma\, r_t\big)\Big);$
return $V,\ \Sigma$

2 Appendix

The following pseudocode is based on the work by [3] on Repulsive Monte Carlo Methods and [2] on Sliced Wasserstein k-means clustering for regime detection.

Algorithm 3: sWk-means clustering with UnifOrtho Projections

Input: Data stream $S \in \mathcal{S}(\mathbb{R}^d)$,;
number of clusters K,;
number of projections L,;
tolerance ϵ for convergence
Output: K cluster assignments and corresponding 1d barycenters

▷ `Preprocessing`
Calculate $l(r^S)$ given $S \in \mathcal{S}(\mathbb{R}^d)$ and define family of empirical distributions $\mathcal{K} = \{\mu_j\}_{1 \le j \le M}$;

▷ `Step 1: UnifOrtho Projection Generation`
$k \leftarrow \lceil L/d \rceil$; ▷ `number of orthogonal bases needed`
$\Theta \leftarrow \emptyset$;
for $i = 1$ **to** k **do**
  Sample a random matrix $Z_i \in \mathbb{R}^{d \times d}$ with i.i.d. $\mathcal{N}(0, 1)$ entries;
  Compute the QR decomposition: $Z_i = Q_i R_i$;
  Let Λ_i be a diagonal matrix where $\Lambda_{i,jj} = \mathrm{sgn}(R_{i,jj})$;
  $U_i \leftarrow Q_i \Lambda_i$; ▷ U_i `is Haar-distributed on` $O(d)$
  Add the d column vectors of U_i to Θ;
$\{\theta^1, \ldots, \theta^L\} \leftarrow \Theta[1 : L]$; ▷ `select exactly` L `directions`

▷ `Step 2: Projection & K-Means Iteration`
Calculate projected distributions $\{\mu'_j(\theta^l)\}_{1 \le j \le M}$ for all $l = 1 \ldots L$;
Initialize cluster assignments $C_1, \ldots, C_K$;
while *convergence criterion* $> \epsilon$ **do**
  for $k = 1$ **to** K **do**
    Compute the 1d Wasserstein barycenter $\bar{\mu}_k(\theta^l)$ for each projection l using empirical distributions assigned to C_k;
  for $j = 1$ **to** M **do**
    Assign μ_j to the cluster C_{k^*} where $k^* = \arg\min_k \sum_{l=1}^{L} W_p^p\big(\mu'_j(\theta^l), \bar{\mu}_k(\theta^l)\big)$;
return $C_1, \ldots, C_K$ *and barycenters* $\{\bar{\mu}_k(\theta^l)\}$

References

[1] B. Horvath, A. Muguruza, and M. Tomas. Wasserstein k-means for clustering distributions. *Journal of Machine Learning Research*, 2024.

[2] Qinmeng Luan and James Hamp. Automated regime classification in multidimensional time series data using sliced wasserstein k-means clustering. *Data Science in Finance and Economics*, 5(3):387–418, 2025.

[3] Rémi Bardenet Vladimir Petrovic and Agnès Desolneux. Repulsive monte carlo on the sphere for the sliced wasserstein distance. *arXiv preprint arXiv:2509.10166*, 2025.